

STTAURX

Security Hardening Proposal

Based on Third-Party Security Audit

Document Status	PROPOSAL - Awaiting Approval
Date	February 3, 2026
Priority	CRITICAL (Must address before production)
Source	Third-Party Security Audit

Executive Summary

An independent security audit identified **7 vulnerabilities** in the current database implementation plan. This document provides a detailed breakdown of each issue and proposes specific fixes that must be implemented before production deployment.

Overall Assessment: "The infrastructure is generally robust for a Phase 1 deployment, particularly in its use of modern security standards like Prisma ORM for SQL injection prevention and Bcrypt for password hashing." However, critical gaps exist that could lead to data loss, identity theft, and audit tampering.

Vulnerability Summary

#	Severity	Issue	Risk
1	CRITICAL	Mock Data Leakage Risk	Data loss on restart
2	MEDIUM	Single Point of Failure	DoS vulnerability
3	MEDIUM	JWT Revocation Missing	Persistent attacker access
4	MEDIUM	RBAC Not Granular	Privilege escalation
5	MEDIUM	Bcrypt Cost Too Low	Password cracking risk
6	CRITICAL	KYC Documents Unencrypted	Identity theft
7	MEDIUM	Audit Logs Mutable	Tampering risk

Issue #1: Mock Data Leakage Risk (CRITICAL)

Audit Finding: The hybrid architecture uses a USE_DATABASE flag to switch between mock and production modes. If this flag is misconfigured in the .env file during a production deployment, the system could revert to in-memory mock data, leading to a massive data loss event where real transactions are 'stored' in volatile memory and vanish upon a server restart.

The Problem:

- If .env file is missing or corrupted, defaults to mock mode
- If someone sets USE_DATABASE=false by mistake, data loss occurs
- No warning or safeguard in production

Proposed Fix: Hardcode production mode + startup validation

```
if (NODE_ENV === 'production') { USE_DATABASE = true; // Forced }
Database connection check at startup - fail fast if no DB
```

Issue #6: KYC Document Storage (CRITICAL)

Audit Finding: The plan does not specify if sensitive files (Passports, IDs) are stored as encrypted blobs in the database or as plain files on the droplet. Storing them unencrypted on a single droplet is a major compliance risk for a financial platform.

Proposed Fix: AES-256-GCM Encrypted Storage

Component	Description
encryptedContent	AES-256-GCM encrypted document (Bytes)
encryptionIv	Initialization Vector - 12 bytes
authTag	GCM authentication tag - 16 bytes

Issue #3: JWT Revocation Complexity

Audit Finding: The document does not explicitly define a 'Blacklist' or 'Revocation' mechanism for compromised tokens. Without this, a stolen 7-day refresh token could allow an attacker persistent access until it naturally expires.

Proposed Fix: Redis Token Blacklist + Token Versioning

- revokeToken(tokenId) - Add specific token to blacklist
- revokeAllUserTokens(userId) - Increment version to invalidate all tokens
- checkTokenBlacklist middleware - Verify before allowing access

Issue #4: RBAC Specificity

Audit Finding: The Phase 1 schema does not include a roles or permissions table. This suggests roles might be hardcoded (e.g., user.isAdmin = true), which is a common vulnerability that leads to 'Privilege Escalation' if the Boolean logic is bypassed.

Proposed Fix: Enhanced Role System with Granular Permissions

Role	Purpose
CLIENT	Standard user (importer/exporter)
ORACLE	Logistics Agent / Digital Notary
AUDITOR	Read-only oversight (audit logs)
ADMIN	Platform administrator
SUPER_ADMIN	Emergency system control (Council)

Issue #5: Bcrypt Cost Factor

Audit Finding: A cost of 12 is currently acceptable (4,096 iterations), but for a gold-backed financial platform in 2026, it is on the lower end of the high-security spectrum. Modern standards often recommend 13 or 14.

Proposed: Upgrade to cost 14 (~1000ms hash time, 16,384 iterations)

Migration: Re-hash existing passwords on next successful login

Issue #7: Audit Log Mutability

Audit Finding: If the database is compromised via an admin-level SQL account, an attacker could potentially delete their own tracks from this table. For a 'Gold Stablecoin Platform,' these logs should ideally be mirrored to an immutable 'Sovereign Node' to ensure they cannot be tampered with.

Proposed Fix: Hash-Chained Audit Logs (Mini-Blockchain)

- Each entry contains SHA-256 hash of previous entry
- PostgreSQL triggers prevent DELETE and hash field modification
- verifyAuditLogIntegrity() function to detect tampering

Implementation Priority

Priority	Issue	Timeline
P0 (CRITICAL)	Mock Data Leakage Fix	Immediate
P0 (CRITICAL)	KYC Document Encryption	Before any KYC data
P1	Bcrypt Cost Upgrade (14)	Before production
P1	Audit Log Immutability	Before production
P1	JWT Revocation System	Before production
P2	Granular RBAC Permissions	Phase 2
P2	Managed Database Migration	Pre-production

Files Created/Modified

File	Description
database/schema.prisma	Security-hardened schema with encryption fields
utils/encryption.util.js	AES-256-GCM encryption utility
utils/security.js	Bcrypt cost 14, hashing helpers
middleware/tokenBlacklist.js	JWT revocation middleware
middleware/rbac.js	Role-based access control
services/auditLog.service.js	Hash-chained audit logging
migrations/audit_triggers.sql	PostgreSQL deletion prevention

Approval Checklist

- ☐ Mock data leakage fix approved
- ☐ KYC encryption approach approved (AES-256-GCM)
- ☐ Bcrypt cost 14 upgrade approved
- ☐ Audit hash chain implementation approved
- ☐ JWT blacklist approach approved
- ☐ RBAC enhancement timeline approved (Phase 2)
- ☐ Managed DB migration timeline approved

Document generated: February 04, 2026 at 01:19

Based on Third-Party Security Audit

STTAURX Gold Stablecoin Platform